

JAVA BACKEND OS SERIES

운영체제 핵심 정리

Java 백엔드 개발자를 위한

STEP 0 · ROADMAP

전체 학습 지도 · 선수지식 · 진단 30문항 · 핵심 용어 100개

독자 기준 전자공학 전공, C# 제조·검사 장비 4년, Java 입문 수준에서 Java/Spring 제조 데이터 플랫폼 백엔드로 전환

기준 Java 21 이상 | 공통 도메인 FactoryPulse 실시간 장비·검사 데이터 플랫폼

판 1.0 · 2026-08-26

SERIES STANDARD

이 시리즈의 제작 현장

Step 1~9에서도 바뀌지 않는 공통 규격

항목	확정 규격
언어/버전	한국어, Java 21 이상. Preview 기능은 반드시 상태와 JDK 버전을 표기
판형/출력	A4 세로, 18 mm 좌우 여백, 한국어 글꼴 사용, 흑백 인쇄 가능
구조	각 Step 독립 독해. 공통 20개 장 구성. 중복은 줄이고 'Step N, 장·절'로 교차 참조
시각 언어	개념=회색, 경고=굵은 테두리+'주의', 면접=이중선+'면접', 구현차이=점선+조건 라벨
코드	기본 패키지 com.factorypulse, Java 문법 강조, 80자 안팎, 잘못된 코드→장애→수정 코드
근거	공식/권위 자료 우선. 언어 명세·JVM 구현·OS 구현을 분리하고 변동 가능성을 명시
검수	PDF와 DOCX를 전 페이지 이미지 렌더링. 글꼴·잘림·빈 페이지·목차·페이지 번호 검사
한 문장 원칙 '왜 느려졌는가'를 OS → JVM → 프레임워크 → 애플리케이션 순서로 추적할 수 있게 만든다.	

CONTENTS

목차

정적 목차: 이 판의 PDF 페이지와 일치하도록 검수

장	내용	페이지
1	시리즈 제작 규격과 전체 목차	2-9
2	선수지식과 학습 순서	10-13
3	FactoryPulse 공통 시나리오	14-15
4	산출물과 통과 기준	16
5	시작 전 진단 30문항	17-25
6	점수별 학습 방법	26
7	핵심 용어 100개	27-36
8	면접 준비 활용법	37
9	참고문헌 정책과 기준 자료	38-39
10	범위 자체 검토와 한 페이지 요약	40-42

읽는 순서 처음이라면 4→10→17→26→27 순서로 읽고, 경험이 있다면 17쪽 진단부터 시작한다.

01 · ORIENTATION

이 책을 사용하는 법

읽기보다 '설명·실행·관찰'의 반복이 핵심

1. 먼저 진단

17-22쪽을 자료 없이 풀고, 23-25쪽 해설로 채점한다.

2. 약한 축을 표시

OS, 동시성, 메모리, I/O, JVM/Spring 다섯 축 중 60% 미만인 축을 표시한다.

3. 매주 산출물 생성

요약 노트가 아니라 실행 가능한 코드, 측정 로그, 장애 설명을 남긴다.

4. 설명의 층위를 분리

Java가 보장하는 것, JVM이 구현하는 것, Linux가 수행하는 것을 섞지 않는다.

5. 면접식으로 마감

개념 정의→내부 동작→장애 증상→확인 도구→대안과 비용 순으로 2분 답변을 만든다.

주의 '용어를 안다'와 '운영 중 증상을 원인에 연결한다'는 다르다. 모든 Step의 통과 기준에는 관찰 증거가 포함된다.

01 · ORIENTATION

당신의 경험을 새 경력으로 연결하기

이미 가진 경험	백엔드/OS로 번역	이번 시리즈에서 확장
C# 장비 제어	스레드, 소켓, 버퍼, 예외 복구 경험	Java 스레드/JMM/NIO/JVM 관찰
CIM-TCP 통신	연결 상태, 재전송, 메시지 경계	FD, 커널 버퍼, epoll, 백프레서
검사 이미지 Review	대용량 I/O와 메모리 압력	페이지 캐시, Direct Buffer, GC 외 메모리
OpenCV	네이티브 라이브러리·버퍼 경계	JNI/오프heap의 개념적 위험과 계측
백준 실버	자료구조·복잡도 감각	동시성·I/O·자원 상한이 있는 시스템 사고

의도적으로 다시 배울 것

- 프로세스·스레드·주소 공간을 '정의'가 아니라 관측 가능한 자원으로 보기
- TCP read 한 번이 메시지 한 건이라는 오해 버리기
- 동시성 문제를 실행 순서와 happens-before로 설명하기
- JVM 힙만 보지 않고 스레드 스택·메타스페이스·Direct Memory·커널 버퍼까지 보기

02 · PRODUCTION SPEC

전 단계 공통 20개 장 구조

전반부	후반부
1. 학습 목표	2. 선수지식 확인
3. 한 문장 핵심 정의	4. 개념 설명
5. 운영체제 내부 동작	6. Java/JVM 연결
7. 제조 데이터 플랫폼 사례	8. 정상 코드
9. 잘못된 코드와 장애	10. 수정 코드
11. 구조도 또는 실행 시간측	12. 자주 하는 오해
13. 실무 확인 방법	14. 면접 질문과 답변
15. 예상 꼬리 질문	16. 직접 구현 과제
17. 복습 문제	18. 정답과 해설
19. 한 페이지 요약	20. 참고문헌

교차 참조 규칙 현재 Step의 이해에 필요한 최소 정의만 3~5문장으로 재소개하고, 상세 설명은 'Step N, 장·절'을 가리킨다.

02 · PRODUCTION SPEC

용어·그림·박스·출처의 통일 규칙

표기	의미	예시
[언어 명세]	모든 적합한 Java 구현이 지켜야 할 규칙	JLS의 happens-before
[JVM 명세]	class 파일·실행 모델의 추상 규칙	JVMS 런타임 데이터 영역
[HotSpot 구현]	특정 JVM 구현과 버전의 동작	G1 기본값, 가상 스레드 스케줄러 세부
[Linux 구현]	Linux 커널/API의 동작	epoll, cgroup v2, /proc
[Spring 계약]	프레임워크가 문서화한 동작	Graceful Shutdown 설정
[조건부]	버전·플랫폼·구현체에 따라 달라짐	‘현재 OpenJDK 21 HotSpot/Linux에서’

박스 라벨

- 핵심: 반드시 한 문장으로 기억할 정의
- 주의: 장애 또는 오해로 연결되는 경계조건
- 면접: 60–120초 답변 빠대
- 실무 확인: 명령·메트릭·프로파일링 증거
- 구현 차이: 명세와 특정 구현을 분리

03 · SERIES MAP

전체 시리즈 목차 ①

Step 0-4: 실행 단위, 메모리, 동시성

Step	제목	핵심 절	예상 쪽
0	로드맵·선수지식·진단·용어집	학습 방법, 30문항 진단, 100용어, 공통 시나리오	40-48
1	Kernel-System Call-Process-Thread-Scheduling	모드 전환, 생성/종료, 문맥 교환, 상태, CPU 스케줄링, Java Thread	55-65
2	Process Memory-Virtual Memory-Paging-TLB	주소 공간, 페이지 테이블, fault, TLB, RSS, 스택/힙/네이티브 연결	55-65
3	Race Condition-동기화-Java Memory Model	원자성/가시성/순서성, 락, volatile, synchronized, happens-before	60-70
4	Thread Pool-Deadlock-Async-Virtual Thread	큐/포화/백프레서, 교착, Future, 비동기, Java 21 가상 스레드	55-65

범위 경계 Step 4의 구조적 동시성은 Java 21에서 Preview였음을 표시하고 개념 비교까지만 다룬다. 가상 스레드는 Java 21 정식 기능으로 다룬다. [R3]

03 · SERIES MAP

전체 시리즈 목차 ②

Step 5-9: I/O, JVM, Spring, 운영, 종합 설계

Step	제목	핵심 절	예상 쪽
5	I/O-Interrupt-FD-Socket-TCP-Java NIO	장치→커널→프로세스, FD, 버퍼, TCP 스트림, epoll, Selector	65-75
6	JVM Memory-GC-STW-Direct Buffer	할당, 런타임 영역, GC root, G1/ZGC 개요, pause, 오프heap	55-65
7	Spring 요청-DB Pool-Graceful Shutdown	요청 생명주기, 서블릿 스레드, HikariCP, 타임아웃, 종료 순서	55-65
8	Linux-Docker-자원 제한-관측성-성능	/proc, cgroup v2, 컨테이너 한계, 메트릭/로그/트레이스, JFR/perf	65-75
9	제조 데이터 수집 종합 사례-면접	용량 산정, 장애/복구, 부하 시험, 설계 리뷰, 면접 문제	65-77

전체 예상 분량: 약 570-670쪽. Step별 분량은 코드·그림·해설 밀도에 따라 ±5쪽 조정할 수 있으나 범위 경계는 유지한다.

04 · PREREQUISITES

각 Step의 선수관계

선수관계: 왼쪽에서 오른쪽으로



그림 1. 권장 선수관계. 화살표는 '완료해야만 시작 가능'이 아니라 '이해 비용을 낮추는 순서'를 뜻한다.

분기	가능한 우회
네트워크 경험이 강함	Step 1 후 Step 5를 먼저 보고, Step 2~4로 돌아와도 된다.
Java 동시성이 약함	Step 1→3→4 순서를 고정한다.
운영 경험이 급함	Step 1·2 핵심 요약 후 Step 8 실습을 병행하되 Step 6을 건너뛰지 않는다.

05 · TEN-WEEK PLAN

10주 학습 순서 ①

주 8~10시간 기준: 개념 35%, 구현 40%, 관찰·설명 25%

주	학습	실습 산출물	통과 질문
1	Step 0 + Step 1	프로세스/스레드 상태 관찰 노트	시스템 콜과 메서드 호출 차이는?
2	Step 2	메모리 지도와 page fault 관찰	RSS와 Java heap은 왜 같지 않은가?
3	Step 3	경쟁 상태 재현 + 수정 테스트	volatile이 원자성을 주지 않는 예는?
4	Step 4	bounded queue 수집기	포화 시 어떤 요청을 늦추거나 버릴까?
5	Step 5 전반	프레이밍이 있는 TCP 수신기	read()와 메시지 경계는 왜 다른가?

주간 루프 월: 개념 지도 → 화: 정상 코드 → 수: 장애 재현 → 목: 계측 → 금/주말: 2분 설명과 복습 문제

05 · TEN-WEEK PLAN

10주 학습 순서 ②

주	학습	실습 산출물	통과 질문
6	Step 5 후반	NIO 기반 다중 장비 연결 실험	Selector는 무엇을 줄이고 무엇을 못 줄이나?
7	Step 6	할당/GC/JFR 리포트	STW와 CPU 100%는 같은 현상인가?
8	Step 7	Spring 수집 API + DB pool 실험	요청·스레드·커넥션의 병목 순서는?
9	Step 8	Docker 제한/관측 대시보드	컨테이너 OOM과 JVM OOM을 어떻게 구분하나?
10	Step 9	종합 설계 문서 + 모의면접	10k 장비 장애를 계층별로 설명할 수 있는가?

시간이 부족할 때

- 필수선: Step 0→1→2→3→5→6→7→8→9
- Step 4는 스레드 풀 포화·교착·가상 스레드 절을 우선
- 각 주의 '관찰 증거'와 '통과 질문'은 생략하지 않는다

운영체제·JVM·Spring의 관계도

운영체제 · JVM · Spring: 책임의 층

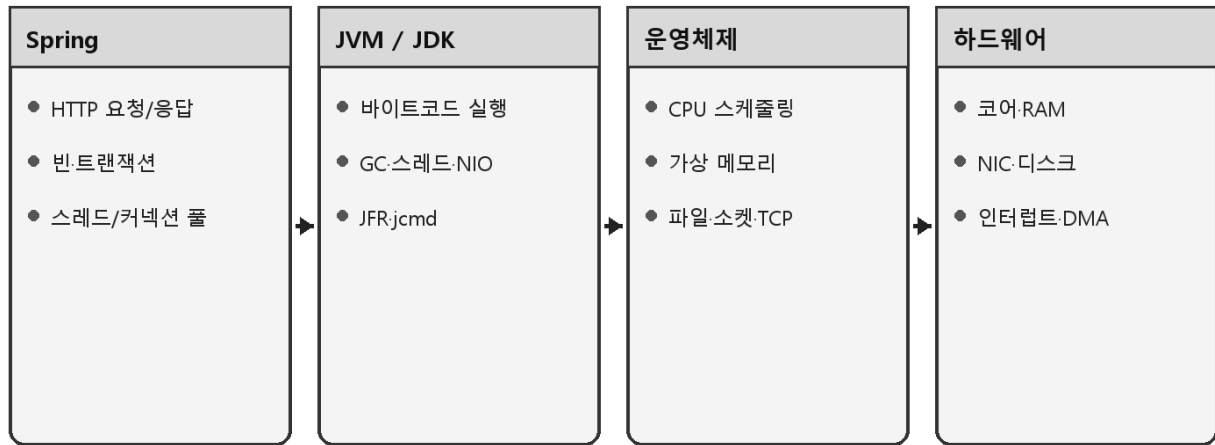


그림 2. 아래 계층은 자원을 제공하고, 위 계층은 정책과 추상화를 더한다. 실제 요청은 네 계층을 왕복한다.

구분 JLS는 Java 프로그램의 의미를 규정하고, JVMJS는 JVM의 추상 구조를 규정한다. HotSpot과 Linux는 그 규칙을 만족하는 구체 구현이다. [R1][R2]

FactoryPulse: 전 단계 공통 도메인

FactoryPulse 공통 시나리오

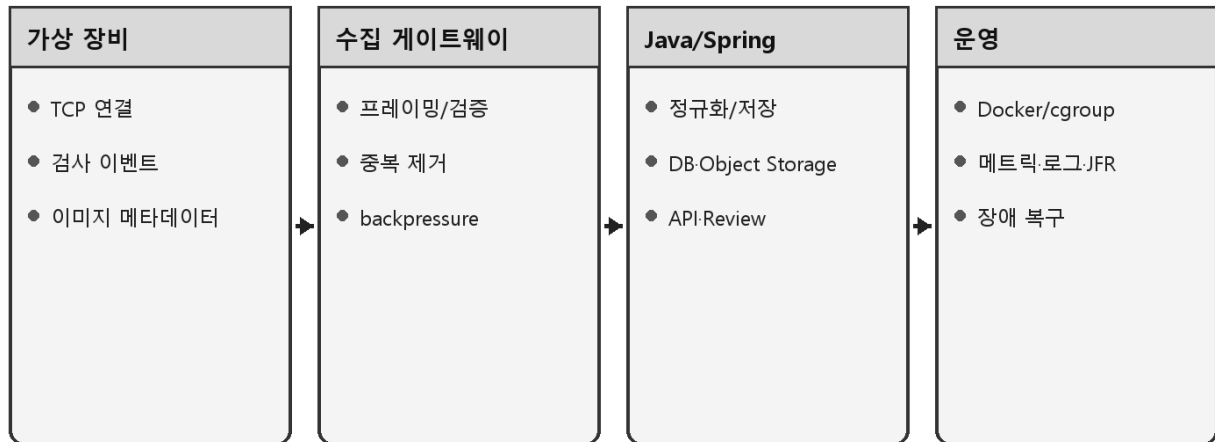


그림 3. 가상 장비가 TCP로 검사 이벤트를 보내고 Java/Spring 플랫폼이 수집·저장·조회한다.

고정 가정	값
장비	1,000대에서 시작, 장기 목표 10,000대
이벤트	길이-prefix 바이너리 프레임 + JSON 메타데이터
이미지	원본은 Object Storage, 이벤트에는 URI·해시·크기
신뢰성	at-least-once 전송을 가정하고 eventId로 중복 제거
SLO 예시	수집 성공률 99.9%, 정상 시 p99 저장 지연 2초 이내

07 · COMMON SCENARIO

이벤트 계약과 실행 시간축

```
package com.factorypulse.protocol;
public record InspectionEvent (
String eventId, String equipmentId, long capturedAt,
int resultCode, String imageUrl, long imageBytes ) { }
```

시간	주체	행동	관찰 지점
t0	장비	TCP connect	SYN/accept queue, 연결 수
t1	장비	길이 4B + payload 전송	recv buffer, partial read
t2	수집기	프레임 조립·검증	queue depth, parse error
t3	서비스	중복 확인·DB 저장	pool wait, transaction time
t4	서비스	ACK 전송	send buffer, timeout
t5	Review	이미지 URI 조회	object latency, cache

주의 TCP는 바이트 스트림이다. send 1회와 read 1회의 경계가 보존된다고 가정하지 않는다. 상세는 Step 5, 'TCP 스트림과 애플리케이션 프레임링' 참조. [R7]

08 · PASS CRITERIA

각 단계의 산출물과 통과 기준

Step	산출물	통과 기준
0	진단표·10주 계획·개인 용어 카드	약한 축 2개와 학습 순서를 근거와 함께 결정
1	상태/스케줄링 관찰 노트	프로세스·스레드·시스템 콜을 2분 설명
2	메모리 지도·fault 실험	heap/RSS/native 차이를 측정값으로 설명
3	race 재현 테스트	happens-before로 실패와 수정을 설명
4	포화 가능한 수집기	큐 상한·거부·타임아웃 정책을 선택
5	TCP/NIO 수집기	부분 읽기·재연결·backpressure 테스트 통과
6	JFR/GC 분석서	pause·할당률·native memory를 구분
7	Spring+DB 부하 시험	요청/스레드/커넥션 병목과 종료 순서 검증
8	컨테이너 관측 리포트	cgroup 한계와 JVM/OS 지표를 상관 분석
9	종합 설계+모의면접	장애 3종 재현, 용량 근거, 20문항 구두 답변

09 · DIAGNOSTIC

시작 전 진단 1-5

자료를 보지 말고 각 1점, 핵심어가 맞으면 정답

1. [OS-01] 프로그램과 프로세스의 차이를 한 문장으로 쓰라.

답: _____

2. [OS-02] 사용자 모드 코드가 파일을 읽기 위해 커널 기능을 요청하는 경계는 무엇인가?

답: _____

3. [OS-03] 문맥 교환 시 바뀌는 상태 두 가지를 쓰라.

답: _____

4. [OS-04] 동시성과 병렬성은 같은가? 이유를 쓰라.

답: _____

5. [OS-05] 스레드마다 독립적인 대표 메모리 영역은 무엇인가?

답: _____

채점 원칙 부분점수 없이 0/1점. 자신 있게 설명하지 못했다면 0점으로 두어 약점을 과소평가하지 않는다.

09 · DIAGNOSTIC

시작 전 진단 6-10

자료를 보지 말고 각 1점, 핵심어가 맞으면 정답

6. [MEM-01] 가상 주소와 물리 주소를 연결하는 핵심 구조는?

답: _____

7. [MEM-02] 페이지 폴트는 항상 오류인가?

답: _____

8. [MEM-03] TLB가 줄이는 비용은 무엇인가?

답: _____

9. [MEM-04] RSS와 JVM 최대 힙이 같지 않은 이유를 두 가지 쓰라.

답: _____

10. [MEM-05] swap이 늘 때 지연시간 꼬리가 길어질 수 있는 이유는?

답: _____

09 · DIAGNOSTIC

시작 전 진단 11-15

자료를 보지 말고 각 1점, 핵심어가 맞으면 정답

11. [CON-01] `count++`가 여러 스레드에서 안전하지 않은 이유는?

답: _____

12. [CON-02] 가시성과 원자성의 차이를 쓰라.

답: _____

13. [CON-03] `volatile`이 복합 연산을 원자적으로 만드는가?

답: _____

14. [CON-04] 교착상태의 필요조건 네 가지 중 두 가지를 쓰라.

답: _____

15. [CON-05] `bounded queue`가 운영 안정성에 주는 이점은?

답: _____

09 · DIAGNOSTIC

시작 전 진단 16-20

자료를 보지 말고 각 1점, 핵심어가 맞으면 정답

16. [IO-01] TCP read 한 번이 송신 메시지 한 건과 일치하는가?

답: _____

17. [IO-02] 파일 디스크립터가 가리킬 수 있는 객체 두 가지를 쓰라.

답: _____

18. [IO-03] blocking I/O에서 스레드가 CPU를 계속 사용하는가?

답: _____

19. [IO-04] epoll/Selector가 해결하는 문제를 한 문장으로 쓰라.

답: _____

20. [IO-05] TCP 흐름 제어와 애플리케이션 backpressure의 차이는?

답: _____

09 · DIAGNOSTIC

시작 전 진단 21-25

자료를 보지 말고 각 1점, 핵심어가 맞으면 정답

21. [JVM-01] JLS와 JVM의 역할 차이는?

답: _____

22. [JVM-02] GC가 회수하는 것은 '참조되지 않는 객체'인가, '도달 불가능한 객체'인가?

답: _____

23. [JVM-03] STW가 발생하면 모든 OS 프로세스가 멈추는가?

답: _____

24. [JVM-04] DirectByteBuffer 용량은 Java heap만 보면 충분한가?

답: _____

25. [JVM-05] Java 21 가상 스레드는 OS 스레드와 항상 1:1인가?

답: _____

09 · DIAGNOSTIC

시작 전 진단 26-30

자료를 보지 말고 각 1점, 핵심어가 맞으면 정답

26. [SPR-01] 요청 스레드 수를 늘리면 DB 처리량이 항상 증가하는가?

답: _____

27. [SPR-02] DB 커넥션 풀 대기의 대표 지표는?

답: _____

28. [SPR-03] SIGTERM을 받은 서버의 graceful shutdown 목표는?

답: _____

29. [OPS-01] 컨테이너 메모리 제한과 호스트 메모리는 같은가?

답: _____

30. [OPS-02] CPU 사용률만으로 지연 원인을 확정할 수 있는가?

답: _____

09 · DIAGNOSTIC

진단 정답과 해설 1-10

1. **OS-01** 프로그램은 저장된 코드/데이터이고 프로세스는 OS가 자원을 부여해 실행 중인 인스턴스다.
2. **OS-02** 시스템 콜 경계다.
3. **OS-03** 예: 프로그램 카운터/레지스터, 스택 포인터, 스케줄링 상태.
4. **OS-04** 아니다. 동시성은 겹친 진행을 다루고 병렬성은 실제 동시 실행이다.
5. **OS-05** 스레드 스택(및 레지스터 실행 문맥).
6. **MEM-01** 페이지 테이블이며 TLB가 최근 변환을 캐시한다.
7. **MEM-02** 아니다. demand paging의 정상 경로일 수 있고 유효하지 않은 접근이면 오류다.
8. **MEM-03** 가상→물리 주소 변환에서 페이지 테이블 접근 비용.
9. **MEM-04** 힙 밖의 스택·메타스페이스·코드 캐시·Direct Memory와 공유 라이브러리/매핑 등이 있기 때문이다.
10. **MEM-05** 디스크 I/O를 동반한 page-in이 예측하기 어려운 대기시간을 만든다.

09 · DIAGNOSTIC

진단 정답과 해설 11-20

- 11. **CON-01** 읽기-수정-쓰기의 여러 단계가 서로 끼어들 수 있기 때문이다.
- 12. **CON-02** 가시성은 다른 스레드의 쓰기를 관측하는 문제, 원자성은 연산이 중간 상태 없이 한 단위로 보이는 문제다.
- 13. **CON-03** 아니다. volatile은 가시성/순서 보장을 제공하지만 일반적인 복합 연산의 원자성은 보장하지 않는다.
- 14. **CON-04** 상호 배제, 점유 대기, 비선점, 순환 대기 중 둘.
- 15. **CON-05** 메모리 사용 상한을 만들고 포화 시 거부·지연·드롭 정책을 명시할 수 있다.
- 16. **IO-01** 아니다. TCP는 바이트 스트림이므로 분할·병합될 수 있다.
- 17. **IO-02** 예: 일반 파일, 소켓, 파이프, 이벤트 객체.
- 18. **IO-03** 대개 커널에서 대기하며 실행 가능한 CPU 시간을 소비하지 않지만 전환/깨움 비용은 있다.
- 19. **IO-04** 많은 I/O 대상의 준비 상태를 적은 스레드로 감시한다.
- 20. **IO-05** TCP 흐름 제어는 수신 측 커널 버퍼 보호, 애플리케이션 backpressure는 전체 처리 파이프라인의 생산률 제어다.

09 · DIAGNOSTIC

진단 정답과 해설 21-30

- 21. **JVM-01** JLS는 Java 언어 의미, JVMs는 JVM 추상 기계와 class 파일/실행 규칙을 정의한다.
- 22. **JVM-02** GC root에서 도달 불가능한 객체다.
- 23. **JVM-03** 아니다. 해당 JVM의 Java 실행 스레드가 safepoint 등에서 멈추는 현상이다.
- 24. **JVM-04** 아니다. 오프heap/네이티브 메모리와 프로세스 전체 RSS를 함께 봐야 한다.
- 25. **JVM-05** 아니다. 가상 스레드는 JDK가 더 적은 carrier 플랫폼 스레드 위에 스케줄할 수 있다. [R3]
- 26. **SPR-01** 아니다. DB 커넥션·DB CPU/락이 병목이면 대기만 늘 수 있다.
- 27. **SPR-02** connection acquisition time, pending/waiting count, timeout 수.
- 28. **SPR-03** 새 작업 수락을 줄이고 진행 중 작업을 제한 시간 내 완료·정리한 뒤 종료하는 것.
- 29. **OPS-01** 아니다. cgroup 제한은 컨테이너가 실제로 사용할 수 있는 상한이다.
- 30. **OPS-02** 아니다. I/O 대기, 락, 큐, GC, 외부 서비스 등과 함께 봐야 한다.

10 · SCORING

진단 점수별 학습 방법

총점	해석	학습 방법
0-9	기초 재구축	10주 순서를 그대로 수행. Step 1-3은 코드보다 그림과 관찰을 먼저 한다.
10-17	개념 파편화	약한 축을 먼저 표시. 각 Step의 선수지식·오해·실무 확인을 생략하지 않는다.
18-23	실무 연결 필요	정의 복습은 빠르게, 장애 재현·관측·면접 설명에 시간을 더 배정한다.
24-27	상급 진입	틀린 축만 보강하고 Step 7-9의 용량/장애 설계 비중을 높인다.
28-30	검증 중심	각 Step 복습 문제로 바로 이동하되 구현 과제와 통과 기준은 모두 수행한다.

축별 경고선

축	문항	보강
OS	1-5	Step 1
메모리	6-10	Step 2
동시성	11-15	Step 3-4
I/O	16-20	Step 5
JVM/Spring/운영	21-30	Step 6-8

한 축에서 60% 미만이면 총점과 무관하게 해당 Step을 순서대로 학습한다.

11 · GLOSSARY

핵심 용어 1-10

정의는 이 시리즈의 일관된 사용 기준이며, 구현 조건은 본문에서 별도 표기

#	용어	정의
1	커널	하드웨어 자원과 프로세스를 관리하며 시스템 콜 서비스를 제공하는 OS 핵심.
2	사용자 모드	권한이 제한된 애플리케이션 실행 모드.
3	커널 모드	특권 명령과 커널 자원 접근이 가능한 실행 모드.
4	시스템 콜	사용자 프로그램이 커널 서비스를 요청하는 규격화된 진입점.
5	프로세스	독립 주소 공간과 OS 자원을 가진 실행 인스턴스.
6	스레드	프로세스 안의 실행 흐름과 스케줄링 단위.
7	PCB	OS가 프로세스 상태·식별자·문맥 등을 보관하는 구조의 일반명.
8	문맥 교환	CPU가 실행 대상을 바꾸며 실행 문맥을 저장·복원하는 과정.
9	스케줄러	실행 가능한 작업 중 CPU를 배정할 대상을 선택하는 OS 구성요소.
10	타임 슬라이스	선점형 스케줄링에서 작업에 허용하는 연속 실행 시간 단위.

11 · GLOSSARY

핵심 용어 11-20

정의는 이 시리즈의 일관된 사용 기준이며, 구현 조건은 본문에서 별도 표기

#	용어	정의
11	선점	OS가 실행 중 작업의 CPU를 회수할 수 있는 성질.
12	run queue	CPU에서 실행될 준비가 된 작업의 대기 구조.
13	CPU-bound	대부분 시간을 계산에 쓰는 작업.
14	I/O-bound	대부분 시간을 I/O 완료 대기에 쓰는 작업.
15	동시성	여러 작업의 진행이 시간상 겹치도록 구성하는 성질.
16	병렬성	여러 작업이 실제로 같은 시각에 실행되는 성질.
17	인터럽트	장치나 CPU가 실행 흐름에 비동기 사건을 알리는 메커니즘.
18	예외/트랩	현재 명령 실행과 관련해 동기적으로 제어가 커널로 이동하는 사건.
19	시그널	Unix 계열 OS가 프로세스에 사건을 알리는 메커니즘.
20	SIGTERM	정상 정리를 기대하며 종료를 요청하는 대표 시그널.

11 · GLOSSARY

핵심 용어 21-30

정의는 이 시리즈의 일관된 사용 기준이며, 구현 조건은 본문에서 별도 표기

#	용어	정의
21	주소 공간	프로세스가 볼 수 있는 논리적 메모리 주소의 범위.
22	가상 메모리	가상 주소를 물리 메모리·파일 등에 매핑하는 추상화.
23	물리 메모리	RAM에 실제 존재하는 메모리 자원.
24	페이지	가상 메모리를 관리하는 고정 크기 블록.
25	페이지 프레임	물리 메모리에서 페이지를 담는 고정 크기 블록.
26	페이지 테이블	가상 페이지와 물리 프레임/상태를 연결하는 구조.
27	페이지 폴트	현재 변환/접근을 즉시 완료할 수 없어 OS 처리가 필요한 사건.
28	minor fault	디스크 읽기 없이 해결 가능한 페이지 폴트.
29	major fault	보통 저장장치 I/O가 필요한 페이지 폴트.
30	TLB	최근 주소 변환을 캐시하는 CPU 구조.

11 · GLOSSARY

핵심 용어 31-40

정의는 이 시리즈의 일관된 사용 기준이며, 구현 조건은 본문에서 별도 표기

#	용어	정의
31	RSS	프로세스 페이지 중 현재 RAM에 상주한 양의 지표.
32	swap	메모리 압력 시 페이지를 보조 저장장치에 보관하는 영역/동작.
33	copy-on-write	복사본을 공유하다 쓰기 시점에 실제 복사하는 기법.
34	mmap	파일/객체를 프로세스 주소 공간에 매핑하는 인터페이스.
35	stack	호출 프레임 지역 변수 등을 담는 스레드별 메모리 영역.
36	heap	동적 할당 객체가 주로 위치하는 메모리 영역.
37	native memory	JVM 프로세스가 Java heap 밖에서 사용하는 메모리의 총칭.
38	memory leak	더 이상 필요 없지만 회수되지 않아 사용량이 지속 증가하는 현상.
39	OOM	요청한 메모리를 확보하지 못한 상태/오류.
40	cgroup memory limit	Linux cgroup이 작업 집합에 허용하는 메모리 상한.

11 · GLOSSARY

핵심 용어 41-50

정의는 이 시리즈의 일관된 사용 기준이며, 구현 조건은 본문에서 별도 표기

#	용어	정의
41	race condition	실행 타이밍에 따라 결과가 달라지는 결함 조건.
42	data race	동기화되지 않은 충돌 접근이 존재하는 JMM상의 조건.
43	원자성	연산이 관찰 가능한 중간 상태 없이 한 단위로 취급되는 성질.
44	가시성	한 스레드의 쓰기를 다른 스레드가 관측할 수 있는 성질.
45	순서성	연산 순서에 대해 관찰자가 의존할 수 있는 보장.
46	임계 구역	공유 상태를 일관되게 다뤄야 하는 코드 구간.
47	mutex	한 번에 하나의 실행 주체만 임계 구역에 들어가게 하는 락.
48	semaphore	허용 수만큼 동시 접근을 제어하는 동기화 도구.
49	monitor	상호 배제와 조건 대기를 결합한 동기화 추상화.
50	condition variable	조건이 충족될 때까지 락과 함께 대기/통지하는 도구.

11 · GLOSSARY

핵심 용어 51-60

정의는 이 시리즈의 일관된 사용 기준이며, 구현 조건은 본문에서 별도 표기

#	용어	정의
51	happens-before	JMM에서 가시성과 허용 실행을 판단하는 순서 관계. [R1]
52	synchronized	Java monitor 획득/해제를 사용하는 언어 수준 동기화 구성.
53	volatile	JMM의 가시성-순서 규칙을 제공하는 필드 한정자.
54	CAS	기대값과 같을 때만 값을 바꾸는 원자적 비교-교환 연산.
55	deadlock	작업들이 서로 필요한 자원을 기다려 영원히 진행하지 못하는 상태.
56	livelock	상태는 바뀌지만 서로 양보하며 유효한 진전이 없는 상태.
57	starvation	특정 작업이 자원을 장기간 배정받지 못하는 상태.
58	thread pool	작업 실행을 위해 재사용되는 제한된 스레드 집합.
59	bounded queue	최대 용량이 정해진 대기열.
60	backpressure	소비 능력에 맞춰 생산-수락 속도를 제어하는 정책.

11 · GLOSSARY

핵심 용어 61-70

정의는 이 시리즈의 일관된 사용 기준이며, 구현 조건은 본문에서 별도 표기

#	용어	정의
61	Future	비동기 결과의 완료·값·실패를 표현하는 Java 추상화.
62	CompletableFuture	단계 조합과 완료 처리를 제공하는 Java Future 구현/API.
63	platform thread	일반적으로 OS 스레드에 1:1로 매핑되는 Java 스레드. [R4]
64	virtual thread	JDK가 carrier 위에 스케줄하는 경량 java.lang.Thread. Java 21 정식. [R3]
65	carrier thread	가상 스레드의 Java 코드를 실제 실행하는 플랫폼 스레드.
66	pinning	가상 스레드가 blocking 중 carrier를 놓지 못하는 구현상 상황의 일반명.
67	structured concurrency	관련 동시 작업의 생명주기를 한 작업 단위로 다루려는 모델. Java 21 API는 Preview.
68	blocking	작업이 조건/자원 완료까지 진행을 멈추는 동작.
69	non-blocking I/O	준비되지 않은 I/O가 호출 스레드를 장기 대기시키지 않는 방식.
70	asynchronous I/O	완료를 나중에 통지받는 I/O 모델.

11 · GLOSSARY

핵심 용어 71-80

정의는 이 시리즈의 일관된 사용 기준이며, 구현 조건은 본문에서 별도 표기

#	용어	정의
71	file descriptor	프로세스가 열린 파일·소켓 등 커널 객체를 참조하는 정수 핸들.
72	socket	네트워크 통신 종단점을 나타내는 커널/JDK 추상화.
73	TCP	신뢰성 있는 순서화된 바이트 스트림 전송 프로토콜.
74	UDP	메시지 단위의 비연결형 전송 프로토콜.
75	listen backlog	수신 연결 처리 전 커널이 관리하는 대기열과 관련된 설정/상한.
76	accept	수신 연결에서 새 연결 소켓을 얻는 동작.
77	send buffer	송신 전 데이터를 보관하는 커널 소켓 버퍼.
78	receive buffer	수신 후 애플리케이션이 읽기 전 데이터를 보관하는 커널 소켓 버퍼.
79	epoll	Linux의 확장 가능한 I/O 준비 상태 통지 인터페이스.
80	Selector	Java NIO에서 여러 Channel의 준비 상태를 선택하는 추상화.

11 · GLOSSARY

핵심 용어 81-90

정의는 이 시리즈의 일관된 사용 기준이며, 구현 조건은 본문에서 별도 표기

#	용어	정의
81	ByteBuffer	Java NIO의 바이트 저장·position/limit/capacity 모델.
82	DirectByteBuffer	네이티브 I/O와의 복사 최적화를 위해 off-heap 메모리를 사용할 수 있는 버퍼.
83	JVM	Java bytecode를 실행하고 런타임 서비스를 제공하는 추상 기계/구현.
84	JLS	Java 언어의 문법·타입·실행 의미를 정하는 명세.
85	JVMS	class 파일과 JVM 추상 실행 모델을 정하는 명세.
86	HotSpot	OpenJDK에서 널리 쓰이는 JVM 구현.
87	bytecode	JVM 명령 집합으로 표현된 class 파일 코드.
88	JIT compiler	실행 중 자주 쓰는 bytecode를 네이티브 코드로 컴파일하는 구성요소.
89	safepoint	JVM이 특정 전역 작업을 위해 스레드 상태를 안정적으로 다룰 수 있는 지점/프로토콜.
90	GC root	도달 가능성 분석의 시작점이 되는 참조 집합.

11 · GLOSSARY

핵심 용어 91-100

정의는 이 시리즈의 일관된 사용 기준이며, 구현 조건은 본문에서 별도 표기

#	용어	정의
91	garbage collection	도달 불가능 객체의 메모리를 자동 회수하는 런타임 작업.
92	STW	JVM 작업을 위해 Java 애플리케이션 스레드 실행을 일시 정지하는 구간.
93	allocation rate	단위 시간당 새로 할당되는 객체 메모리 양.
94	promotion	객체가 수집기 세대/영역 정책에 따라 더 오래 사는 영역으로 이동하는 것.
95	G1 GC	힙을 region으로 나눠 pause 목표를 지향하는 HotSpot 수집기.
96	ZGC	낮은 pause를 목표로 대부분의 작업을 동시 수행하는 HotSpot 수집기.
97	metaspace	HotSpot이 클래스 메타데이터에 사용하는 native memory 영역.
98	code cache	JIT 컴파일된 기계어 등을 저장하는 JVM 메모리 영역.
99	JFR	JDK Flight Recorder. 저부하 이벤트 기록·진단 기능.
100	jcmd	실행 중 JVM에 진단 명령을 보내는 JDK 도구.

12 · INTERVIEW

면접 준비에 활용하는 방법

2분 답변 프레임

정의: 한 문장으로 경계를 긋는다.

내부 동작: OS/JVM/Spring 중 책임 계층을 지정한다.

장애: FactoryPulse에서 나타날 증상과 영향을 말한다.

확인: 메트릭·로그·JFR·Linux 도구 중 증거를 고른다.

대안: 해결책과 비용/부작용을 함께 말한다.

예시 질문

면접 '장비 연결이 1,000대에서 10,000대로 늘자 p99가 급증했습니다. 스레드만 늘리면 해결될까요?'

좋은 답변은 단정 대신 병목 후보를 계층별로 좁힌다: accept/FD 상한 → runnable/blocked 스레드 → 큐 포화 → DB pool wait → GC/메모리 압력. 측정 전에는 원인을 확정하지 않는다.

카드 작성 규칙

앞면	뒷면
질문 + 30초 핵심 정의	내부 동작 3줄 / 장애 증상 2개 / 확인 증거 2개 / 꼬리 질문 2개

13 · REFERENCES

전체 참고문헌 정책

우선순위

1차: JLS/JVMS/JDK API/OpenJDK JEP/Linux man-pages/Spring 공식 문서. 2차: 구현체 문서·권위 있는 교재. 블로그는 재현 보조로만 사용.

주장 단위

버전 의존·성능·구현 세부 주장에 [R#]를 붙인다. 상식적 정의도 혼동 가능성이 높으면 근거를 단다.

층위 분리

[언어 명세], [JVM 명세], [HotSpot 구현], [Linux 구현], [Spring 계약] 라벨을 사용한다.

버전 기록

URL, 문서명, 대상 버전, 열람일을 기록한다. latest 문서는 제품 버전이 바뀌면 다시 검증한다.

불확실성

‘항상’ 대신 조건을 쓴다. 예: ‘OpenJDK 21 HotSpot의 기본 구성에서’, ‘Linux cgroup v2에서’.

코드 검증

예제는 Java 21에서 컴파일/실행하고, OS 의존 실습은 Linux 커널·배포판·컨테이너 조건을 기록한다.

인용 형식 본문 [R3] → 참고문헌의 문서명, 발행 주체, 버전/절, URL, 열람일. 링크가 사라져도 문서 식별이 가능해야 한다.

13 · REFERENCES

Step 0 기준 참고문헌

이후 Step은 여기에 세부 문서를 추가하고 기준 정책을 재사용

[R1] Oracle, Java Language Specification SE 21, Chapter 17 Threads and Locks
<https://docs.oracle.com/javase/specs/jls/se21/html/jls-17.html> (열람 2026-08-26)

[R2] Oracle, Java Virtual Machine Specification SE 21
<https://docs.oracle.com/javase/specs/jvms/se21/html/> (열람 2026-08-26)

[R3] OpenJDK, JEP 444: Virtual Threads (Delivered, JDK 21)
<https://openjdk.org/jeps/444> (열람 2026-08-26)

[R4] Oracle, Java SE 21 API: java.lang.Thread
<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html> (열람 2026-08-26)

[R5] Linux man-pages project, proc(5), signal(7), epoll(7)
<https://man7.org/linux/man-pages/> (열람 2026-08-26)

[R6] Spring Boot Reference, Graceful Shutdown
<https://docs.spring.io/spring-boot/reference/web/graceful-shutdown.html> (열람 2026-08-26)

[R7] IETF, RFC 9293: Transmission Control Protocol (TCP)
<https://www.rfc-editor.org/rfc/rfc9293> (열람 2026-08-26)

[R8] OpenJDK, JDK 21 Project
<https://openjdk.org/projects/jdk/21/> (열람 2026-08-26)

[R9] Oracle, Java SE 21 Tools: jcmd/jfr
<https://docs.oracle.com/en/java/javase/21/docs/specs/man/> (열람 2026-08-26)

[R10] Linux kernel documentation, cgroup v2
<https://docs.kernel.org/admin-guide/cgroup-v2.html> (열람 2026-08-26)

14 · SCOPE REVIEW

중복·누락·과도한 범위 자체 검토

점검	판정	조정
Step 1 vs 3 스레드 중복	통제	Step 1은 실행 단위/스케줄링, Step 3은 공유 메모리/JMM.
Step 4 vs 5 비동기 중복	통제	Step 4는 작업·큐·생명주기, Step 5는 I/O readiness와 NIO.
Step 2 vs 6 메모리 중복	통제	Step 2는 OS 주소 변환, Step 6은 JVM 할당/GC/오프heap.
Step 6 vs 8 성능 분석	통제	Step 6은 JVM 진단, Step 8은 OS/컨테이너 상관 분석.
백프레서 누락 위험	보완	Step 4 큐 정책, Step 5 네트워크 적용, Step 9 통합 검증.
종료·복구 누락 위험	보완	Step 7 종료 순서, Step 8 시그널/컨테이너, Step 9 장애 복구.
네트워크 범위 과다	축소	TCP 혼잡 제어 수학·라우팅은 제외; 서버 장애 분석에 필요한 수준만.
GC 범위 과다	축소	수집기 내부 알고리즘은 비교 원리까지만; 튜닝은 증거 기반 절차 중심.
분산시스템 범위 확대	제외	Kafka 합의·DB 복제 이론은 인터페이스/병목 관점만 언급.

결론 Java 백엔드 운영에 필요한 OS 핵심을 유지하면서, 분산시스템·DB 내부·네트워크 전공 범위로 확장되는 것을 막았다.

15 · ONE-PAGE SUMMARY

한 페이지 요약

질문	기억할 답
왜 이 순서인가?	실행 단위→메모리→동시성→I/O→JVM→Spring→컨테이너 순으로 원인 층을 쌓기 위해.
공통 사례는?	가상 장비가 TCP로 검사 이벤트를 보내는 FactoryPulse 플랫폼.
무엇을 만들어야 하나?	각 Step마다 실행 코드, 장애 재현, 관찰 증거, 2분 면접 답변.
무엇을 구분해야 하나?	Java 언어 보장, JVM 명세, HotSpot 구현, Linux 구현, Spring 계약.
어떻게 시작하나?	30문항 진단→축별 60% 확인→10주 계획 보정→Step 1.
완료의 기준은?	용어 암기가 아니라 증상에서 계층별 원인을 좁히고 증거로 확인하는 능력.

오늘 할 일

- 17-22쪽 진단을 25분 안에 푼다.
- 26쪽에서 약한 축을 두 개 표시한다.
- 10-12쪽 선수관계와 10주 계획에 개인 일정을 적는다.
- 용어집에서 낯선 용어 15개를 카드로 만든다.
- Step 1 시작 전 Linux/JDK 21 실행 환경을 준비한다.

16 · HANDOFF

Step 1 진입 체크리스트

이번 요청의 제작 범위는 여기까지이며 Step 1 PDF는 생성하지 않음

Java 21 이상에서 작은 프로그램을 컴파일·실행할 수 있다.

Linux 셸에서 프로세스 목록과 종료 코드를 확인할 수 있다.

FactoryPulse 이벤트 계약의 필드를 설명할 수 있다.

진단 점수와 약한 축 두 개를 기록했다.

주 8~10시간 중 최소 3시간을 실행·관찰에 배정했다.

‘Java가 보장’과 ‘HotSpot이 구현’을 구분해 말할 준비가 됐다.

통과 기준 6개 중 5개 이상이면 Step 1로 진행한다. 환경 항목이 미완료면 먼저 JDK 21과 Linux 실습 환경을 준비한다.

Step 1 예고

첫 질문은 ‘Java 메서드가 실행될 때 CPU·커널·프로세스·스레드는 각각 무엇을 하는가?’이다. 시스템 콜, 문맥 교환, 스케줄링을 FactoryPulse TCP 수집기의 실행 시간축 위에서 연결한다.